



Generacion deCodigo con Inteligencia Artificial

Ejercicio Practico · Modulo 2

Nivel Intermedio · Requiere VS Code y herramienta de IA

Nombre del alumno: ANGEL DAVID ECHENIQUE VILLANUEVA

Fecha: 27/02/26

Instrucciones

Este ejercicio requiere que uses **VS Code** y una herramienta de IA (Copilot, ChatGPT, Gemini...) El objetivo no es solo que el codigo funcione: debes **analizar, mejorar y justificar** cada decision.

Entrega: capturas de pantalla + respuestas escritas en este documento.

B1

BLOQUE 1

Lenguajes y estrategias — En la practica

Ya sabemos que lenguajes soporta mejor la IA y por que. Ahora vamos a comprobarlo en directo.

Ejercicio practico — Abre VS Code para completar esta tarea

Ejercicio 1.1 — Compara el soporte real por lenguaje

Prompt de partida

"Crea una funcion en [LENGUAJE] que valide un numero de telefono espanol en formato +34 seguido de 9 digitos. Incluye docstring, type hints y al menos 3 casos de prueba."

Usa la misma tarea en dos lenguajes distintos: pide a la IA que cree una funcion que valide un numero de telefono espanol (formato +34 XXXXXXXXXX).

Ejercicio 1.1 — (continuacion) Tabla comparativa

Lenguaje 1 (ej. Python)

Lenguaje 2 (ej. JavaScript)



Modulo 2 · Generacion deCodigo con IA · Dicampus

Nivel Intermedio · Requiere VS Code y herramienta de IA



¿El codigo compila/ejecuta sin errores? Si.	SI
¿Incluye type hints o tipado? SI type hints nativos.	No tiene tipado nativo al menos que usemos typeScript
¿Los casos de prueba son correctos? Si.	Si.
¿La documentacion es clara? Si, utilizando docstrings.	Si, utilizamos JSDoc para documentar.
Calidad general (1-5) 5	5
¿Que diferencias notaste entre los dos lenguajes? ¿Cual funciona mejor y por que?	
Los tipados en python usa type hints nativo (numero: str -> bool) en cambio javaScript no tiene tipado nativo al menos que use typeScript. Disintas formas para documentar, Py puede documentar con docstrings y JS permite documentar con JSDoc. Para ejecutarlos vemos una diferencia, Py usa importaciones y JS usa expresiones regulares no necesita importaciones.	

Ejercicio 1.2 — Aplica la estrategia de Contexto del Proyecto

Vimos que la IA genera mejor codigo cuando conoce el contexto. Prueba la diferencia:

Sin contexto	Con contexto completo
Prompt usado: "Crea un endpoint para login."	Prompt usado: Crea un endpoint de login en Python usando FastAPI. -Usa nombres de funciones descriptivos -Organiza el código por secciones (configuración, modelos, lógica, endpoints) -Agrega docstrings detallados en cada función -Usa JWT para autenticación -Aplica buenas prácticas básicas de seguridad -Incluye instrucciones para ejecutarlo
¿Que genero la IA? Me genero un codigo en lenguaje JavaScript, porque el prompt usado es muy ambiguo le da mucha libertad a la IA para decidir entonces ahora tenemos una conts = usuario que contiene id, email y password.	Me ha generado un endpoint completo en FastAPI, organizado en secciones y haciendo uso de los docstrings descriptivos. En pocas palabras me dio una mini API de backend funcionando para la autenticacion.



Modulo 2 · Generacion deCodigo con IA · Dicampus

Nivel Intermedio · Requiere VS Code y herramienta de IA



¿Fue usable directamente?

No, tuve que instalar las dependencias de js.

No, tuve que instalar dependencias que no tenía, como la instalación de fastapi y uvicorn.

¿Que tuviste que corregir?

Instalaciones de dependencias.
PasswordHash porque se genera cada vez que reinicias.
Y el hash debe guardarse en una base de datos.
Cambios en la variable de entorno JWT, para mayor seguridad. Implementación del try/catch.

Instalación de dependencias porque me daba errores de importaciones.

Problemas con el archivo al no llamarse igual y estar fuera del archivo principal.

Mi prompt con contexto (el que mejores resultados te dio):

El prompt usado:

- 1) Porque le especifique la tecnología a usar Python y le dije que usara una FastAPI.
- 2) Le definí la estructura y le pedí que me organizara el código por secciones aparte le he pedido que documentara cada función con docstrings.
- 3) Le he pedido un mecanismo de autenticación y he incluido niveles de seguridad.

B2

BLOQUE 2

Iteración y refinamiento en VS Code

Una de las estrategias clave del módulo es **no pedir todo de golpe**. Vamos a practicar la generación iterativa construyendo una mini API paso a paso.

Ejercicio práctico — Abre VS Code para completar esta tarea

Endpoints CRUD

Añade los 4 endpoints básicos: GET, POST, PUT, DELETE para tareas.

Prompt exacto que use:

Implementame los 4 endpoints CRUD (GET, POST, PUT, DELETE) para una API REST de gestión de tareas usando FastAPI.

¿Que tuve que corregir o ajustar?



Modulo 2 · Generacion deCodigo con IA · Dicampus

Nivel Intermedio · Requiere VS Code y herramienta de IA



Falta especificar almacenamiento y no definimos el comportamiento esperado.

Ejercicio 2.1 — Construye una mini API en 4 pasos iterativos

Vas a construir una API REST sencilla de gestion de tareas (to-do list) usando el framework que prefieras. Sigue los 4 pasos en orden y anota el prompt exacto:

Tecnologia recomendada

Puedes usar: **FastAPI** (Python), **Express** (Node.js) o **Spring Boot** (Java).

El objetivo es practicar la iteracion, no crear una API compleja.

1 Estructura base

Pide solo la estructura de carpetas y archivos vacios, sin logica.

Prompt exacto que use:

Quiero que me crees únicamente la estructura de carpetas y archivos vacíos para una API REST de gestión de tareas usando FastAPI.

No agregues lógica todavía

¿Que tuve que corregir o ajustar?

Los parametros al pedirle el endpoints. Le eh pedido lo que quiero.

todo_api/

```
|
├── main.py
├── models.py
├── routes.py
└── services.py
```

3 Validacion y errores

Añade validacion de datos y manejo de errores con codigos HTTP correctos.

Prompt exacto que use:

Quiero que me añadas validación de datos usando Pydantic y manejo de errores con códigos HTTP correctos (400, 404, 422).

¿Que tuve que corregir o ajustar?

Quitar validaciones que ya la FastAPI hace automaticamente y asegurarme que los campos obligatorios esten bien definidos.

4 Mejora y optimizacion



Modulo 2 · Generacion deCodigo con IA · Dicampus

Nivel Intermedio · Requiere VS Code y herramienta de IA



Pide a la IA que revise el codigo generado y proponga mejoras.

Prompt exacto que use:

Revisa el código completo de la API y propone mejoras de arquitectura, seguridad y mantenibilidad aplicando el clean code.

¿Que tuve que corregir o ajustar?

Simplificacion de middleware innecesario y mantener la coherencia con el objetivo educativo.

Ejercicio 2.2 — Reflexion sobre la iteracion

¿Que ventajas encontraste al construir la API en pasos vs. pedirlo todo de golpe? ¿Hubo algun paso donde la IA fallo y tuviste que guiarla para que lo resolviste?

Ventajas: Control sobre mi codigo y resultado. Mejor compresion del codigo.

Pedirlo de golpe: Nos da todo en un solo archivo mezclado y puede que funciones pero es una mala practica como programar ya que es mas facil de romper ese codigo.

Fallos: No manejas algunos codigos HTTP 404, put y delete.

Resolviendo fallos: Le indique que codigos HTTP debia devolverme en mi prompt de validaciones y errores.

B3

BLOQUE 3

Tecnicas avanzadas — Validacion y alternativas

Dos de las tecnicas avanzadas mas utiles son **pedir a la IA que revise su propio codigo** y **pedir varias alternativas para comparar**.

Ejercicio practico — Abre VS Code para completar esta tarea

¿Que errores o mejoras detecto la IA? ¿Estas de acuerdo? ¿Hay algo que ella no detecto pero tu si?

Errores: Falta de manejo explicito de tipos, validaciones vagas de parametros y mejor uso de excepciones específicas.

De acuerdo: Mejorar robustez porque hace el codigo mas mantenible y reduce errores en el futuro.

No Detecto: no sugiero test automaticos, no valido que tipo_cliente fuera un valor permitido aparte no considero normalizacion de tipo_clientes EJ: MAYUSCULAS - minusculas.

Ejercicio 3.1 — Validacion automatica del codigo generado

Pide a la IA que genere esta funcion y luego aplica la tecnica de **Validacion Inmediata**:

Funcion: calcular_descuento(precio, porcentaje, tipo_cliente) - Si tipo_cliente es 'premium': descuento adicional del 5%
- Si precio < 0 o porcentaje fuera de [0,100]: lanzar excepcion



Modulo 2 · Generacion deCodigo con IA · Dicampus

Nivel Intermedio · Requiere VS Code y herramienta de IA



- Devolver precio final redondeado a 2 decimales

Prompt de generacion que use:

- quiero que me generes una funcion en python lamda calcular_descuento(precio, porcentaje, tipo_cliente) que:
- Si tipo_cliente es "premium", aplique un descuento adicional del 5%.
- Si precio es menor que 0 o porcentaje está fuera del rango [0,100], lance una excepción.
- Devuelva el precio final redondeado a 2 decimales.
- Usa type hints y docstring.

Prompt de validacion que use (pide a la IA revisar su propio codigo):

Revisa el código anterior como si fueras un revisor senior de Python.

- Detecta posibles errores lógicos.
- Evalúa validaciones.
- Propón mejoras de robustez y claridad.
- Señala posibles problemas en producción.
- Sugiere optimizaciones si son necesarias.

Ejercicio 3.2 — Pide 3 alternativas y elige la mejor

Pide a la IA tres implementaciones distintas de una funcion que busque duplicados en una lista: simple, optimizada y funcional. Analiza cada una:



Criterio	Version simple	Version optimizada	Version funcional
Complejidad temporal (O(?))	O(n²)	O(n)	O(n)
¿Es facil de leer?	Muy facil	Media	Alta
¿Maneja lista vacia?	SI	SI	SI
¿Maneja tipos mixtos? Implementacion	Puede fallar si no son hashbles .	No si hay tipos nos hashables.	Depende de
¿La usarias en produccion?	NO	SI	SI, con revision.

¿Cual elegirías y por que? Justifica tecnicamente tu decision:

Por lo que entendi eligiria la optimizada, porque no introduce complejidad innecesaria y para produccion es como la m
Priorizamos la claridad y mantenibilidad sobre estilo funcional.

B4

BLOQUE 4

Workflow humano-IA — Proyecto completo

Vimos que el workflow profesional tiene **5 fases**. En este bloque vas a aplicarlas todas en un mini proyecto real.

Ejercicio practico — Abre VS Code para completar esta tarea

Ejercicio 4.1 — Mini proyecto: Sistema de login con JWT

Desarrolla un sistema de autenticacion basico usando el **workflow humano-IA**. Usa el framework que prefieras (FastAPI, Express, Spring Boot).

Fase	Prompt que use	¿Que genero la IA?	¿Que ajuste yo?
Diseno	Creame un diseño de sistemas de autentificacion con JWT en FastAPI. Dame una arquitectura basica del proyecto y los endpoints necesario No quiero codigo.	➤ Modelo de usuario. ➤ Endpoint de login. ➤ Funcion para generar JWT.	Añadi capas de servicios. Y revision de lo creado.
Implementacion	Quiero que me organices el sistema anterior usando modulacion, mejorando la nomenclatura y añadiendo docstrings. Tambien implementa el endpoint/login en FastAPI	Endpoint funcional. Generacion de JWT Hash de contraseñas	Separe la logica de autentificacion y mejor control de errores.
Refactorizacion	Revisa el codigo como senior developer y dime mejoras de clean code y seguridad .	Me sugiero separar dependencias, mejoras en funciones y añadir manejo explicito de excepciones.	Organice imports, simplifique validaciones y elimine mucho codigo redundante.

Fase	Prompt que use	¿Que genero la IA?	¿Que ajuste yo?
Testing	Generame test unitarios con pytest para el sistema de login.	Test de login valido, de contraseña incorrecta y test de token valido	Correccion de fixtures y mock de base de datos.
Documentacion	Quiero un readme bien documentado explicando como ejecutar la API	Intrucciones de con uvicorn, instalacion de dependencias y explicacion de endpoint.	Añadi ejemplos reales de request/response y añadi seguridad.

Ejercicio 4.2 — Patrones de prompts por lenguaje

El modulo explica que cada lenguaje tiene sus convenciones. Escribe un prompt adaptado a tu lenguaje que cumpla con todos los elementos caracteristicos:



Modulo 2 · Generacion deCodigo con IA · Dicampus

Nivel Intermedio · Requiere VS Code y herramienta de IA



Mi prompt adaptado al lenguaje del proyecto:

Creame un endpoint de autenticacion de FastAPI siguiendo PEP8, usando pydantic para la validacion, sepame por c proyecto y usa docstrings.
Manejame excepciones http y jwt con expiracion configurable por variable de entorno.

¿Que diferencia notaste entre usar un prompt generico y uno adaptado? ¿El codigo encajaba mejor en el estilo del proyecto?

Muchisima, porque el generico me da el codigo todo mezclado y es muy dificil entenderlo.
Cuando lo adaptamos y el damos parametros a seguir a la IA, el codigo tiene una arquitectura donde reparte responsab una buena nomenclatura y bien documentado con docstrings.

B5

BLOQUE 5

Reto de integracion

Este bloque integra todo lo visto. Construye un componente completo aplicando al menos **6 estrategias del modulo** de forma consciente.

Ejercicio practico — Abre VS Code para completar esta tarea

Ejercicio 5.1 — Construye un modulo de auditoria de accesos

Descripcion del reto

Crea un modulo que registre cada intento de login (usuario, timestamp, IP, exito/fallo).

Debe permitir consultar el historial de un usuario y detectar intentos fallidos consecutivos.

Usa el lenguaje del proyecto anterior. Aplica al menos **6 estrategias del modulo**.



Modulo 2 · Generacion deCodigo con IA · Dicampus

Nivel Intermedio · Requiere VS Code y herramienta de IA



```
Python

from dataclasses import dataclass
from datetime import datetime
from collections import defaultdict, deque
from typing import List

# -----
# 1 Modelo de dominio
# -----

@dataclass
class LoginAttempt:
    username: str
    timestamp: datetime
    ip: str
    success: bool

# -----
# 2 Servicio de auditoría
# -----

class AccessAudit:

    def __init__(self):
        # Diccionario + acceso O(1) por usuario
        self._logs = defaultdict(deque)

    # Registrar intento
    def register_attempt(self, username: str, ip: str, success: bool) -> None:
        attempt = LoginAttempt(
            username=username,
            timestamp=datetime.now(),
            ip=ip,
            success=success
        )
        self._logs[username].append(attempt)

    # Obtener historial completo
    def get_user_history(self, username: str) -> List[LoginAttempt]:
        return list(self._logs[username])

    # Detectar intentos fallidos consecutivos
    def has_consecutive_failures(self, username: str, threshold: int = 3) -> bool:
        count = 0

        # Recorremos desde el final (más reciente)
        for attempt in reversed(self._logs[username]):
            if not attempt.success:
                count += 1
                if count >= threshold:
                    return True
            else:
                break

        return False
```



Modulo 2 · Generacion deCodigo con IA · Dicampus

Nivel Intermedio · Requiere VS Code y herramienta de IA



 Ejemplo de uso

```
</> Python

audit = AccessAudit()

audit.register_attempt("juan", "192.168.1.10", False)
audit.register_attempt("juan", "192.168.1.10", False)
audit.register_attempt("juan", "192.168.1.10", False)

print(audit.has_consecutive_failures("juan")) # True
```

Ejercicio 5.2 — Documenta tu proceso

Rellena esta tabla indicando que estrategias aplicaste y como las usaste:

Estrategia aplicada	¿Como la use en este ejercicio?
1.Contexto completo	Especifiqué que usaba FastAPI y JWT
2. Iteracion progresiva	Construir el modulo de auditoria en pasos.
3.Validación inmediata	Le pedí revisión tipo senior
4.Solicitar alternativas	Pedí distintas formas de registrar logs
5.Refinamiento	Eliminé middleware innecesario
6.Revisión crítica humana	Detecté problemas que la IA no vio



Ejercicio 5.3 — Reflexion final

¿En que momento del ejercicio la IA te sorprendio (positiva o negativamente)? ¿Hubo algo que no pudo resolver correctamente y tuviste que arreglar tu? ¿Que aprendiste sobre los limites de la IA generando codigo?

Me respondio todo pero eh verificado con los mismo prompt y preguntas en 3 IA y cada una da una respuesta distinta.

Igualmente genero estructuras completas y funcionales muy rapido.

Tuvo algunas limitaciones en casos reales, no me sugiero los test tuve que pedirselos y se le olvida si el codigo puede ser escalable en el futuro.

A mi me toco organizar arquitectura y en la seguridad(las variables de entorno), manejo de correcto de errores http y me di cuenta que la IA en sintaxis puede ser 99% fiable pero en diseño y seguridad puede fallar.

Checklist de entrega — Antes de entregar, comprueba que has completado todo:

- ☒ Ejercicio 1.1: tabla comparativa de lenguajes completada
- ☒ Ejercicio 1.2: prompt con contexto completo escrito y probado
- ☒ Ejercicio 2.1: API construida en 4 pasos con prompts documentados
- ☒ Ejercicio 3.1: validacion del codigo + analisis de errores detectados
- ☒ Ejercicio 3.2: tabla de 3 alternativas con justificacion de la eleccion
- ☒ Ejercicio 4.1: tabla del workflow completa con los 5 pasos
- ☒ Ejercicio 5.1: modulo de auditoria funcionando en VS Code
- ☒ Ejercicio 5.2: tabla de 6 estrategias documentadas